

1. What is Spring?

Spring is a powerful, open-source Java-based framework used to build scalable, secure, and maintainable enterprise applications.

It provides comprehensive infrastructure support for developing Java applications and promotes best practices like loose coupling, testability, and separation of concerns.

Feature	Description
Open-source	Free to use and backed by a strong community.
Modular	You can use just what you need (e.g., Spring Core, Spring MVC, Spring Boot, etc.).
Lightweight	It's not as heavyweight as older Java EE platforms.
Dependency Injection	Automatically injects dependencies to manage objects more easily.
Aspect-Oriented Programming (AOP)	Helps manage cross-cutting concerns like logging, security, etc.
Integration Ready	Works well with databases, messaging queues, REST APIs, etc.

2. What is Spring Boot?

Spring Boot is a framework built on top of Spring that makes it easy to create stand-alone, production-ready Spring applications — with minimal setup and lots of built-in features.

Feature	Description
Auto-configuration	Automatically configures your app based on the libraries present in your classpath
Embedded servers	No need to deploy to an external Tomcat – it runs with <code>java -jar</code>
Starter dependencies	Bundled dependencies for common tasks (e.g., <code>spring-boot-starter-web</code>)
Production-ready	Built-in monitoring, metrics, and health checks (<code>/actuator</code>)
Opinionated defaults	Sets sensible defaults so you don't have to configure everything manually
Fast prototyping	Perfect for microservices and RESTful APIs development

3. What is the relation between Spring platform and Spring Boot?

Spring Boot = Spring Framework + Auto Configuration + Starter Templates + Embedded Server

- Spring Boot is part of the larger Spring ecosystem. It uses Spring Core, Spring MVC, Spring Data, etc. internally.
- Spring Boot is a layer of abstraction over the Spring Platform that:
- Simplifies project setup with starter templates
- Automatically configures beans and dependencies
- Runs with embedded servers (no need to deploy to Tomcat)
- Spring Boot applications are still Spring applications — they just require less manual configuration and give you more productivity out-of-the-box.

4. What is the relation between Spring platform and Spring framework? **

- Spring Platform is the umbrella term for all Spring projects.
- Spring Framework is the core foundation of the platform.
- Other projects like Spring Boot, Spring Data, Spring Security, etc., depend on Spring Framework for fundamental features like IoC and AOP.

Analogy:

- Spring Platform = ecosystem
- Spring Framework = backbone

5. What is Dependency Injection (DI) and how is it done in Spring?

Dependency Injection is a design pattern where an object's dependencies are provided (injected) by an external source rather than being created by the object itself.

In Spring, DI can be done in three ways:

1. Constructor Injection (Recommended):

```
@Component
public class Service {
    private final Repository repo;
    public Service(Repository repo) {
        this.repo = repo;
    }
}
```

2. Setter Injection:

```
@Component
public class Service {
    private Repository repo;
    @Autowired
    public void setRepository(Repository repo) {
        this.repo = repo;
    }
}
```

3. Field Injection (Not recommended for testing):

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    // ← Field injection: Spring will inject the bean here
    @Autowired
    private PaymentProcessor paymentProcessor;

    public void placeOrder(Order order) {
        // use paymentProcessor without having to pass it in via constructor
        paymentProcessor.process(order.getPaymentDetails());
        // ... other order logic ...
    }
}
```

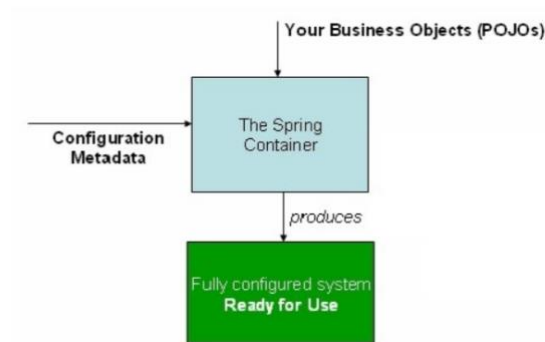
Spring handles DI using its ApplicationContext (or IoC container).

6. What is Inversion of Control (IoC) and how is it related to Spring?

- **Inversion of Control:**
 - The control of object creation and management is handed over to the Spring container.
- **Why it matters:**
 - Instead of you manually creating objects with new, Spring creates and manages them for you.
- **Hollywood Principal**
 - *You don't call Spring; Spring calls your code*

Spring IOC Containers

How does an IoC container works?



How Inversion of Control related to Spring?

- IoC approach (“give me what I need”)
The framework (Spring) is in charge of constructing objects and injecting their dependencies for you. Your classes simply declare what they need.
- IoC Container
 - Spring’s ApplicationContext is the container.
 - It reads bean definitions (via annotations or XML), instantiates beans, wires them together, and manages their full lifecycle.

